

An Analysis of Hyperparameter Optimization in a Supervised Machine Learning Algorithm: K-Nearest Neighbors utilizing the Iris Dataset

Anthony Salva Mijares
Department of Computer Studies &
Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
anthony.mijares@my.jru.edu

I. INTRODUCTION

The primary focus of the study is to build a prediction model capable of classifying iris which are flower species based on their structural or morphological measurements. Utilizing structural or morphological measurements, the prediction model applies a supervised machine learning algorithm to predict which species of iris flower the unknown subject belongs to.

The Iris dataset, which is the dataset employed in this study is a classic as it is often used for machine learning algorithms and visualizations specially for beginners as an introduction. Originally, it was introduced by a British statistician and biologist named, Ronald Fisher through his 1936 published paper “The use of multiple measurements in taxonomic problems”. Inside the whole dataset contains 150 equally distributed sample of sepal and petal measurements, which are both their length and width, from three species namely, Iris setosa, Iris versicolor, and Iris virginica [1].

The algorithm used in this study is the K-Nearest Neighbors (kNN) which is often described as a “lazy learner” as behaves by storing the entire training dataset and utilizes them in making predictions based on the distance between the new data points and existing one [2]. Normally the most commonly used distance measure is the Euclidean, as it measures a straight-line distance between the new and existing data points. In addition, kNN apply the rule of majority voting, based of the highest number of votes for a specific class label and the value of k . In cases where the result is a tie, an implementation can be done by selecting based on the nearer neighbors or through weighing votes based on distance. Furthermore, determining the k value is highly critical in achieving the best accuracy, and the most effective approach is through cross-validation which requires splitting the data into two, training and testing sets evaluating different candidate values of k .

II. METHODOLOGY

In this section, the methodology used to implement the prediction model utilizing K-Nearest Neighbor is described. The whole process is organized and divided into six logical modules, outlining the purpose and procedure of each step.

A. Data Acquisition and Organization

The initial phase of the process involves importing from the python libraries for data visualization which is **matplotlib.pyplot** and the Iris dataset which belongs to scikit-

learn library through **sklearn.datasets**. Thereafter, the **load_iris** function will ensure that the samples are loaded. Once the samples are loaded and retrieved, it will be organized into two structures which are the feature matrix X (**data.data**), which are the measurements and the target vector y (**data.target**), which are the species. This ensures a clear distinction between the variables and the labels.

B. Data Preprocessing

This process involves partitioning the data for training and testing, which utilizes the function **train_test_split** and indicate the **test_size = 0.2**, in which pertains to 20% of 150 samples that will be reserved for testing, hence the remaining 80% or 120 out of 150 will be used to train the model. This ensures the highest accuracy on predictions as most existing studies adopt single, conventional training-testing split ratio which typically 80:20 [3], which can also help prevent overfitting and provides an unbiased evaluation regarding the performance of the model. In addition, the parameter **random_state=0** ensures that the data shuffle will be consistent can be reproduced.

C. Model Training

The third module defines which classifier will be used utilizing the **KNeighborsClassifier** class. On this phase, it set the hyperparameter **n_neighbors** to 5. This will allow the algorithm to look for 5 closest data points and determine the class of an unknown sample through majority voting. Choosing 5 as the hyperparameter will reduce the probability of a tie but not to the point of eliminating it entirely, as they are chances of a 2-2-1 scenario and this is where you need to break the tie by classifying it based on its closest neighbor with the smallest Euclidean distance. Moreover, this module also executes the training phase through **.fit()** will allow the model to learn from the 120 samples partitioned using the features which is X and target vector y .

D. Prediction

The fourth module initiates the model to begin making predictions on the test data utilizing the data learned from training through this command **predictions=knn.predict(X_Test)**

E. Prediction

The fifth module, imports the **accuracy_score** function from the scikit-learn library then evaluates the performance

of the model by checking the accuracy based on the actual values as shown on this command **accuracy = accuracy_score(y_Test, predictions)** and a print command to show the result **print("Accuracy:", accuracy)**. Furthermore, it also imports the **classification_report** function from the same library, **from sklearn.metrics import classification_report**, and prints the result, **print(classification_report(y_Test, predictions))**. The classification report displays the how precised the model predicted a certain class, and how many are correct. It also displays recall, which refers to the amount of true positive over true positive and false negative. F1-score which is the harmonic mean of both precision and recall, and the support which, total number of samples tested. Additionally, it displays both the macro and weighted average.

F. Visualization

The sixth module provides a visual representation of the through scatterplot displaying the predicted and true classes of the test data, **import matplotlib.pyplot as plt** **plt.scatter(X_Test[:,0], X_Test[:,1], c=predictions, cmap='viridis')**. On the other hand, confusion matrix illustrates through a color mapped tabular representation of the number of correct and incorrect predictions for each class, which in this case the darker shades represents a higher count and hints of blue that are not on the diagonal pattern indicates that it was misclassified, **from sklearn.metrics import confusion_matrix** **cm = confusion_matrix(y_Test, predictions)** **plt.imshow(cm, cmap='Blues')**.

III. RESULTS AND EXPERIMENTAL ANALYSIS

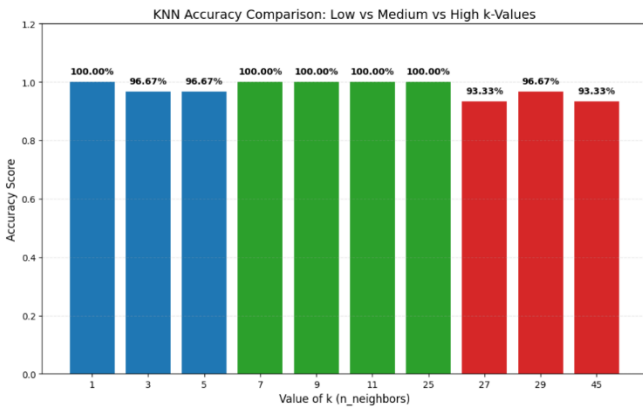


Figure 1. Accuracy Graph

Based on the illustration, Figure 1., setting the hyperparameter by default to 5, the accuracy of how the prediction model performs is at the decent 96.67%, although in comparison in utilizing odd numbers from 7 to 25 will result to a 100% accuracy, and will start to diminish when get passed 27 as it only resulted to 93.33%. On the other hand, lowering it to 3 will not show an improvement although choosing 1 is not really the best option as it is prone to overfitting.

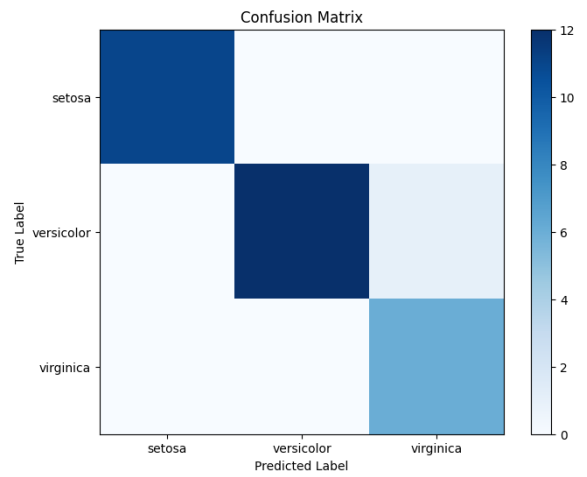


Figure 2. Confusion Matrix

The figure, Confusion Matrix, the Setosa is only white on its own column and its own row, while in contrast both has a faint light blue beside versicolor and on the or virginica indicating that there are errors that happened which is evident on the classification report. Pattern indicates that the model considered the versicolor and virginica too similar and having difficulties for the model to accurately, hence changing the hyperparameter to 7 to 25 may improve the performance in terms of accuracy.

IV. REAL-WORLD APPLICATIONS

A. Government Use Cases for kNN

1) Public Health and Resource Allocation: The Local Government Units (LGUs) and the Department of Health (DoH) utilizes to predict hotspots on where can they prioritize public health and resource allocation and nutritional counseling [4].

2) Land-Use Zoning and Urban Planning: Utilizes kNN to solve and identify uncertainty in soil properties across different location in Valenzuela City by measuring the soil strength and density [5].

3) Science Policy and National R&D Monitoring: Utilize a kNN model automatically classifying and monitor scientific research in space science and its adjacent fields through systematically categorizing vast amounts of scientific literature which can aid funding concerns towards emerging research areas that lack sufficient coverage [6].

B. Technical Specification Audit

An Enhanced K-Nearest Neighbors (KNN) Algorithm for Predicting Child Malnutrition in The City of Manila the study utilizes Euclidean Distance as metric for both their traditional kNN and enhanced models they perform cross-validation across a range of odd k-values between 1 to

11 to compare both their traditional and enhanced kNN algorithm and found out that the enhanced remained stable at high k-values as the traditional declined.

C. Technical Specification Audit

In terms of the black box nature of kNN or the process on how the data goes in and what comes out it is transparent enough for human to understand how did it come up with the decision or prediction compared to more complex deep learning or advanced AI, I might consider trusting the model but still put everything in grain of salt for high-stakes government decision making, so long as algorithms performance are accurate enough and meeting the standards.

V. CONCLUSION

Utilizing the K-Nearest Neighbor (kNN) algorithm on the Iris datasets proves that this so called “lazy learner” can perform accurately when utilizing k hyperparameter from the defaulted 5 resulting to 96.67%, which can be optimized and to 100% accuracy using a odd medium value k , which is between 7 and 25. This perhaps be, also due to the nature of the Iris datasets, which is normally being used as a benchmark, and can be identified through its morphological measurements. Despite, on how effective the kNN in utilizing on simple tasks it requires significant computational power which can be its biggest drawback as due to its nature, storing the entire training dataset to make predictions. Additionally due to its simpler nature compared to other machine learning

which are more complex considering it and having it optimized through cross-validation to ensure reliability and stability can perhaps aid in making hi decision making

REFERENCES

- [1] Fisher, R.A. (1936) *The Use of Multiple Measurements in Taxonomic Problems*. *Annals of Eugenics*, 7, 179-188. <http://dx.doi.org/10.1111/j.1469-1809.1936.tb02137.x>
- [2] Loeloe, M.S., Tabatabaei, S.M., Sefidkar, R. et al. Boosting K-nearest neighbor regression performance for longitudinal data through a novel learning approach. *BMC Bioinformatics* 26, 232 (2025). <https://doi.org/10.1186/s12859-025-06205-1>
- [3] Karabulut, M. Influence of Training–Testing Data Variation on ML-Based Deflection Prediction of GFRP-Reinforced High-Strength Concrete Beams. *Polymers* 2026, 18, 55. <https://doi.org/10.3390/polym18010055>
- [4] Raizen Daguplo, Vivien Agustin and Khatalyn Mata. An Enhanced K-Nearest Neighbors (KNN) Algorithm for Predicting Child Malnutrition in The City of Manila. *ScienceOpen Preprints*. 2024. DOI: 10.14293/PR2199.001360.v1
- [5] Guaña, Dominic & Sace, Arcy Layne & Hilario, Paul & Gumayan, Efren & Perez, Gay. (2023). Performance of genetic algorithm optimized Doc2Vec-kNN for classifying space science and adjacent fields documents with heterogenous sampling.
- [6] Galupino, Joenel & Dungca, Jonathan. (2022). Development of a k-Nearest Neighbor (kNN) Machine Learning Model to Estimate the SPT N-Values of Valenzuela City, Philippines. *IOP Conference Series: Earth and Environmental Science*. 1091. 012021. 10.1088/1755-1315/1091/1/012021.